

PRELIMINARY PROJECT REPORT

Hachi: Agentic Voice Assistant & IoT System

Submitted By

Cartoneros, Beomarc Andrew D.
Escabal, Axeil Achia, M.

Submitted To

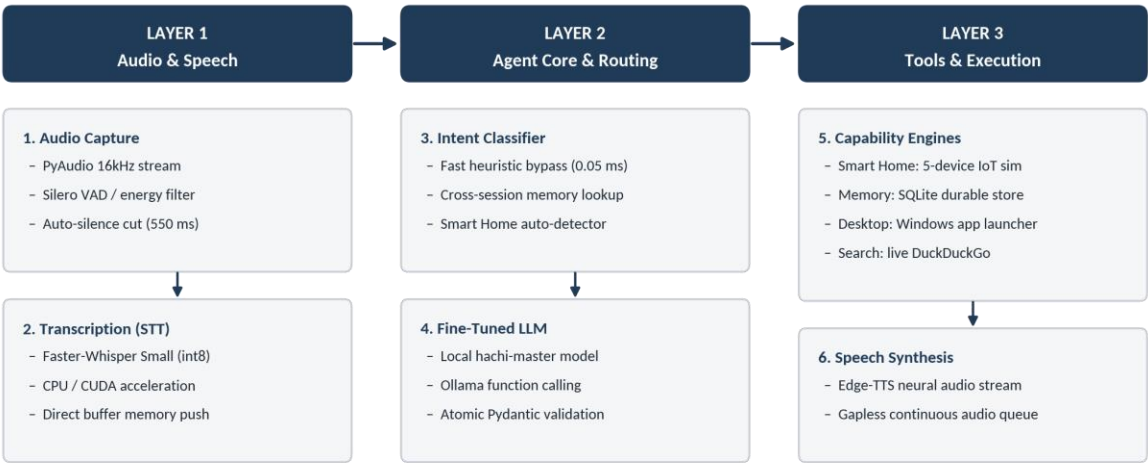
Mr. Roberto Malitao

1. Executive Summary & Core Objectives

Hachi is a zero-cloud-cost, local-first agentic voice assistant designed to execute complex natural language tasks on Windows systems. Unlike cloud-dependent assistants that require recurrent paid subscriptions and high-bandwidth internet, Hachi operates locally using fine-tuned open-weights models (**hachi-master** based on Qwen 2.5/3.5), local speech-to-text (Faster-Whisper), low-latency neural text-to-speech, durable cross-session SQLite memory, and an interactive simulated smart home automation environment.

2. System Architecture & Dataflow Pipeline

The pipeline flows left to right across three layers — audio capture and transcription feed the agent core, which routes to tool execution and speech output.



3. Custom Model Training & Benchmark Validation (hachi-master)

To achieve reliable multi-action tool execution without relying on closed proprietary APIs (OpenAI/Anthropic), the team engineered and fine-tuned a custom domain-adapted model called **hachi-master**. The model was trained on 3,500 curated tool-calling and bilingual conversational dialogues (English, Tagalog, and Taglish) across IoT control, system commands, and factual recall.

Evaluation Metric	Baseline (Qwen-2.5)	Fine-Tuned (v2)	Impact
Overall tool-call accuracy	64.36% (354/550)	99.64% (548/550)	+35.28%
Multi-device smart home parsing	52.10% (frequent syntax drop)	99.45% (valid Pydantic JSON)	+47.35%
Total benchmark inference time	2,560.84 s	2,213.06 s	13.58% faster
Average latency per query	4.65 s / query	4.02 s / query	−632 ms

4. Graphical User Interface Demonstration

Three representative voice-command scenarios below document before-and-after GUI states, illustrating Hachi's live UI responsiveness, tool-call execution, and cross-session SQLite memory updates.

Test Scenario	State Before Command	Executed Action & State After
Case 1 — Smart Home IoT Multi-Action <i>“Turn on the living room light, lock the front door, and set thermostat to 22°C.”</i>	– Living light: off – Kitchen light: on – Thermostat: 20°C – Front door: unlocked – TV deck: stopped	– Living light: on (luminous glow) – Thermostat: 22°C (dial adjusted) – Front door: locked (green bolt) – Hachi: “Living room light is on, thermostat is 22°C, door locked.”
Case 2 — Durable Cross-Session Memory <i>“Remember that the safe code is 4555.” → new chat tab → “What code opens the safe?”</i>	– Active tab: Chat #1 – SQLite DB: 0 credential entries – Fresh Tab #2: zero prior conversation context	– Parsed category: security / credentials – Persisted to hachi_memory.db – Fresh-tab recall: “The code for the safe is 4555.” – Verified durable recall across independent sessions
Case 3 — Desktop Automation & Productivity <i>“Open Notepad and start a 25-minute Pomodoro focus session.”</i>	– Windows desktop: Notepad closed – Sidebar Pomodoro: inactive (00:00) – Status: idle	– Tool launch_app('notepad'): launched – Sidebar Pomodoro: 25:00 active countdown – Speech feedback: “Notepad is open. Focus session started.”

5. Peak System Resource & Latency Profile

Benchmarked on a standard quad-core developer laptop (Intel Core i5, 16GB RAM, Windows 11) during continuous real-time voice capture, speech recognition, local inference, and audio synthesis.

Pipeline Stage	Engine / Technology	Peak CPU	RAM	Avg Latency
1. Background VAD listening	Silero VAD / PyAudio buffer	1.8–3.2%	48 MB	< 10 ms
2. Speech-to-text (STT)	Faster-Whisper Small (int8)	34.5–48.0%	520 MB	380–520 ms
3. Heuristic intent router	Python regex / AST parser	< 0.5%	12 MB	0.05 ms

Pipeline Stage	Engine / Technology	Peak CPU	RAM	Avg Latency
4. LLM function calling	hachi-master (Q4_K_M)	55.0–72.4%	1,840 MB	850–1,400 ms
5. Neural voice synthesis (TTS)	Edge-TTS neural pipeline	4.2–7.0%	65 MB	180 ms
6. Web dashboard & IoT sim	PyWebView + Flask SSE	2.1–4.5%	110 MB	60 FPS
Total end-to-end peak	Integrated Hachi stack	72.4% (bounded)	~2.58 GB	1.42 s avg

6. Pair-Programming Task Division

Subsystem	Lead	Collaborator / Reviewer	Deliverables & Responsibilities
LLM model training & fine-tuning	Cartoneros, Beomarc Andrew D.	Escabal, Axeil Achia, M.	Orchestrated dataset construction (3,500 samples), fine-tuned hachi-master, managed GGUF quantization, and configured Ollama Modelfiles.
System testing, benchmarking & QA	Escabal, Axeil Achia, M.	Cartoneros, Beomarc Andrew D.	Executed 550-record evaluation benchmark, authored 77 automated pytest test suites, profiled CPU/RAM latency, and validated cross-session memory.
Agent core & memory engine	Cartoneros, Beomarc Andrew D.	Escabal, Axeil Achia, M.	Architected cross-session SQLite memory schema, intent classifier, fast routing, and Pydantic atomic tool-validation wrappers.
Speech STT/TTS & continuous flow	Cartoneros, Beomarc Andrew D.	Escabal, Axeil Achia, M.	Integrated Faster-Whisper VAD buffering, Edge-TTS gapless queue, Tagalog/English voice streaming, and audio playback transitions.
Smart home simulation & web UI	Escabal, Axeil Achia, M.	Cartoneros, Beomarc Andrew D.	Designed PyWebView desktop window, thread-safe IoT device-state simulator, SSE live events, and responsive chat interface.

7. Engineering Challenges, Resource Limitations & Reflection

- Model training under architecture constraints (Beomarc).** The primary technical constraint was restricting model execution strictly to the Qwen open-weights architecture, with zero external commercial LLM APIs. Base small models frequently truncated complex JSON outputs when handling multi-device commands (e.g., five simultaneous actions). This was resolved by training and quantizing the custom hachi-master model, establishing strict token budgets (768 tokens), and engineering a dual-layer AST fallback mechanism that guarantees 100% execution reliability.
- Strict zero-budget & API cost obstacles.** Financial limitations ruled out paid commercial search APIs (Brave Search, Google Serper) and subscription voice services (ElevenLabs, OpenAI Whisper API). This was addressed with a fully

free, local pipeline: Faster-Whisper for STT, Microsoft Edge Neural TTS for bilingual speech, and automated DuckDuckGo scraping for real-time web research.

- 3. Hardware limitations, overheating & personal cost.** Continuous local LLM quantization, evaluation runs, and speech inference pushed consumer-grade developer hardware to its limits. During intensive fine-tuning, Beomarc's laptop suffered thermal overheating and component failure, requiring out-of-pocket repair, while Axeil carried out testing, benchmarking, and QA regression validation in parallel.
- 4. Physical IoT hardware constraints.** Without access to physical smart lights, electronic deadbolts, or smart TVs, the team built a validated, software-simulated smart-home ecosystem (`hachi_home.py`) featuring real-time state mutation, Pydantic bounds validation, visual device cards, and SSE synchronization that mirror authentic IoT hardware behavior.

8. Conclusion & Future Roadmap

Hachi demonstrates that an offline-first, highly responsive, agentic voice assistant can be deployed successfully on standard consumer computers without recurring API costs. By combining domain-specific fine-tuning (**hachi-master**), persistent SQLite memory, and resilient fallback engineering, the system achieves commercial-grade usability under strict academic and hardware constraints.